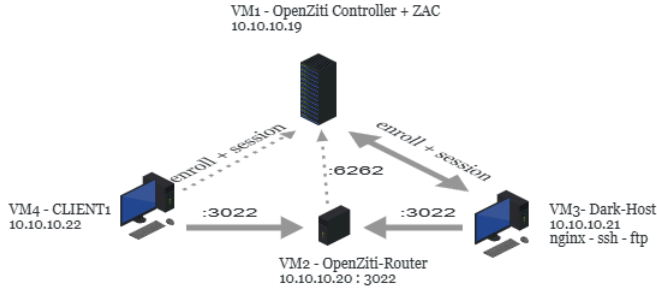


Mimari ve Ağ diyagramları

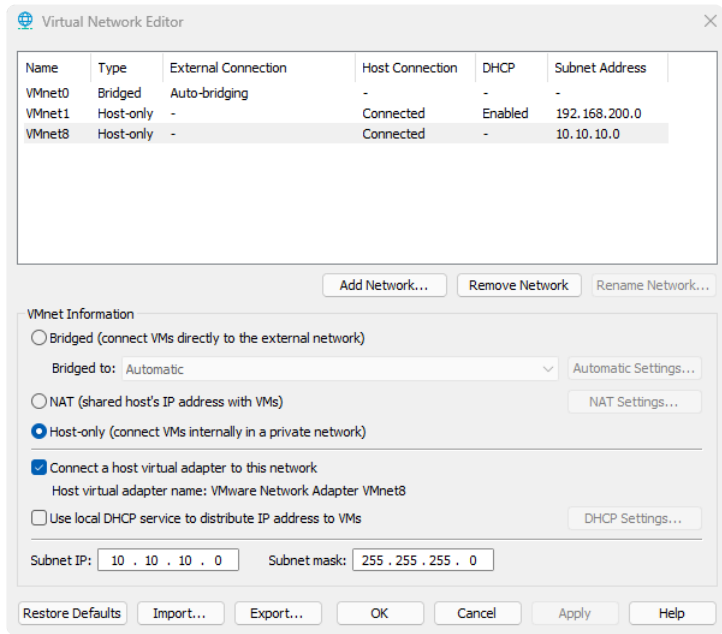
Genel ağ diyagramı



Created in ICOGRAMS

VMWare ağ ayarları görüntüleri

VMWare Workstation network adaptor



VMWare

Tüm makinelerde VMnet8 bağlı

VM1 - OpenZiti Controller

Cancel

Wired

Apply

Details

Identity

IPv4

IPv6

Security

IPv4 Method

☐ Automatic (DHCP)

☐ Link-Local Only

☒ Manual

☐ Disable

☐ Shared to other computers

Addresses

Address

Netmask

Gateway

10.10.10.19

255.255.255.0

10.10.10.2

DNS

Automatic

10.10.10.1

Separate IP addresses with commas

Routes

Automatic

Address

Netmask

Gateway

Metric

☐ Use this connection only for resources on its network

VM1

VM2 - Ziti-Router

Cancel

Wired

Apply

Details

Identity

IPv4

IPv6

Security

IPv4 Method

☐ Automatic (DHCP)

☐ Link-Local Only

☒ Manual

☐ Disable

☐ Shared to other computers

Addresses

Address

Netmask

Gateway

10.10.10.20

255.255.255.0

10.10.10.2

DNS

Automatic

10.10.10.1

Separate IP addresses with commas

Routes

Automatic

Address

Netmask

Gateway

Metric

☐ Use this connection only for resources on its network

VM2

VM3 Dark-Host

Cancel

Wired

Apply

DetailsIdentityIPv4IPv6Security

IPv4 Method

☐ Automatic (DHCP)

☐ Link-Local Only

☒ Manual

☐ Disable

☐ Shared to other computers

Addresses

Address	Netmask	Gateway	
10.10.10.21	255.255.255.0	10.10.10.2	

DNS

Automatic ☒

10.10.10.1

Separate IP addresses with commas

Routes

Automatic ☒

Address	Netmask	Gateway	Metric	

☐ Use this connection only for resources on its network

VM3

VM4 - Client1

İnternet Protokolü Sürüm 4 (TCP/IPv4) Özellikleri

Genel

Ağınız destekliyorsa, IP ayarlarının otomatik olarak atanmasını sağlayabilirsiniz. Aksi halde, IP ayarlarınız için ağ yöneticinize başvurmanız gerekir.

☐ Otomatik olarak bir IP adresi al

☒ Aşağıdaki IP adresini kullan:

IP adresi: 10 . 10 . 10 . 22

Alt ağ maskesi: 255 . 255 . 255 . 0

Varsayılan ağ geçidi: 10 . 10 . 10 . 2

☐ DNS sunucu adresini otomatik olarak al

☒ Aşağıdaki DNS sunucu adreslerini kullan:

Tercih edilen DNS sunucusu: 10 . 10 . 10 . 1

Diğer DNS Sunucusu: 1 . 1 . 1 . 1

☐ Çıkarken ayarları doğrula

Gelişmiş...

Tamam İptal

VM4

Bileşenlerin Online olduğunun görüntüsü

IDENTITIES

Identities

Type to Filter

1-5 of 5

ID	Name	Roles	O/S	SDK	Type	Is A...	Created At	Token	MFA	...
☐	Default Admin			v1.7.0	Default	☒	7/21/2026 ...	--		...
☐	client1	file-user ma		1.18.0	Default		7/22/2026 ...	--		...
☐	dark-host	hosts		v1.7.0	Default		7/23/2026 ...	--		...
☐	openziti-router				Router		7/21/2026 ...	--		...
☐	sdk-app	sdk-hosts		v2.0.0	Default		7/24/2026 ...	--		...

VM1

Enrollment

Identity ve JWT oluşturma

← Edit Identity: client1

FORM ☒ JSON

More Actions

Save

Offline

IDENTITY NAME REQUIRED

client1

SELECT OR CREATE IDENTITY ATTRIBUTES ? OPTIONAL

Add attributes to group Identities

× #file-user

× #manager

× #web-user

AUTH POLICY ? OPTIONAL

Default

EXTERNAL ID OPTIONAL

Optional External ID

SHOW MORE OPTIONS

☐ OFF

ENROLLMENT:

RESET ENROLLMENT

ENABLE / DISABLE:

DISABLE IDENTITY

DISABLE UNTIL:

Indefinite

ASSOCIATED SERVICES ? 4

Type to Filter

ftp-svc
nginx-svc
sdk-zeroneSVC
ssh-svc

ASSOCIATED SERVICE POLICIES ? 4

Type to Filter

file-dial
management-dial
sdk-zeroneSVC-dial-policy

VM1

yeni bir identity nesnesi oluşturulduğunda bir jwt nesnesi indirilebilir şekilde listelendiğindi yerde gözükür.

```
ziti edge create identity sdk-app -o ~/sdk-app.jwt -a "sdk-hosts"
```

aynı şekilde terminalden de yapılabilir.

HEADER, PAYLOAD VE SIGNATURE

bu JWT dosyası 3 parçadan oluşuyor. header, payload ve signature.

Header

header imza algoritmasının özelliklerini yazar

```
{"alg": "RS256", "kid": "6f210eaa2b64dfdf640c006150afd5b6a9a2cce6", "typ": "JWT"}
```

🔗

Payload

payload imzanın geçerliliğini, nerede geçerli olacağını ve tek kullanımlık olduğunun token'ını yazar

```
{  
  "iss": "https://guts-xd:1280",  
  "sub": "6mIvMiw1",  
  "aud": [  
    ""  
  ],  
  "exp": 1788157856,
```

JSON

```
{
  "jti": "dc1434ad-4534-4f47-ba00-32be6fb9b447",
  "em": "ott",
  "ctrls": [
    "tls:guts-xd:6262"
  ]
}
```

Signature

header ve payload bilgilerinin geçerliliğini kanıtlayan en önemli kısım. MÜHÜR

XRF0kFqMnyYQ0Xr-1Hab1iFpQBQNmZQSmRNiv2smrhLFbnjsH89L19QH-0ALUZ5HvmcjrpOMoq8VF01l8c0fiCasFA-i84KRPywvxD1oQ7012bdYT8k55

JWT dosyasını tanımlamak

Bu oluşturulan kimlik tanımlama dosyasına sahip olacak client1 bilgisayarına gidiyoruz diğer adıyla dark-host

Kimlik tanımlama basit aslında.

```
ziti edge enroll ~/sdk-app.jwt -o ~/sdk-app.json
```

Bu komut ile elimizde bu cihaza özel sürekli geçecek bir anahtarımız var. [sdk-app.json](#)

Komutun çıktısı

```
guts@dark-host:~$ ziti edge enroll ~/sdk-app.jwt -o ~/sdk-app.json
INFO    generating 4096 bit RSA key
INFO    enrolled successfully. identity file written to: /home/guts/sdk-app.json
```

[illegible]

VM3

Dosyanın genel görünümü bu şekilde

Oluşturulan anahtarı kullandığımız tek yer

```
ziti tunnel host --identity /home/guts/dark-host.json
```

Bu komut ile openziti-router makinesiyle arasında bir network t nel'i oluřturuluyor.



JSON	Raw Data	Headers
Save	Copy	Collapse All Expand All Filter JSON

ztAPI: "https://guts-xd.1280/edge/client/v1"

VM3

JSON dosyasında nereye istek atılacağına dair bilgi var.

Controller'a ait bir endpoint bu. Sormadan  nce json dosyasında ki sertifika bilgisini veriyor ve kendisini doęruluyor.

Bu endpoint, router'ın adresini s yl yor.

openziti-router:3022 adresine istek atar ve t nel oluřur.

Genel bilgiler

- JWT tek kullanımlıktır.
- Oluřturulan  zel anahtar (json dosyası) hi bir zaman bir yere g nderilmez

SDK uygulama ve  alıřtırma s reci

Uygulamanın kaynak kodu

```
*
Copyright 2019 NetFoundry Inc.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

https://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

*/

package main

import (
    "fmt"
    "github.com/michaelquigley/pfxlog"
    "github.com/openziti/sdk-golang/v2/ziti"
    "github.com/sirupsen/logrus"
    "net"
    "net/http"
    "os"
    "time"
)

type Greeter string

func (g Greeter) ServeHTTP(resp http.ResponseWriter, req *http.Request) {
    var result string
```

```

if name := req.URL.Query().Get("name"); name != "" {
    result = fmt.Sprintf("Hello, %v, from %v\n", name, g)
    fmt.Printf("Saying hello to %v, coming in from %v\n", name, g)
} else {
    result = "Who are you?\n"
    fmt.Println("Asking for introduction")
}
if _, err := resp.Write([]byte(result)); err != nil {
    panic(err)
}
}

func serve(listener net.Listener, serverType string) {
    if err := http.Serve(listener, Greeter(serverType)); err != nil {
        panic(err)
    }
}

func httpServer(listenAddr string) {
    listener, err := net.Listen("tcp", listenAddr)
    if err != nil {
        panic(err)
    }
    fmt.Printf("listening for non-ziti requests on %v\n", listenAddr)
    serve(listener, "plain-internet")
}

func zitifiedServer() {
    options := ziti.ListenOptions{
        ConnectTimeout: 5 * time.Minute,
        MaxConnections: 3,
    }

    // Get identity config
    cfg, err := ziti.NewConfigFromFile(os.Args[1])
    if err != nil {
        panic(err)
    }

    // Get service name (defaults to "simpleService")
    serviceName := "simpleService"
    if len(os.Args) > 2 {
        serviceName = os.Args[2]
        fmt.Printf("Using the provided service name [%v]", serviceName)
    } else {
        fmt.Printf("Using the default service [%v]", serviceName)
    }

    ctx, err := ziti.NewContext(cfg)

    if err != nil {
        panic(err)
    }

    listener, err := ctx.ListenWithOptions(serviceName, &options)
    if err != nil {
        fmt.Printf("Error binding service %v\n", err)
        panic(err)
    }

    fmt.Printf("listening for requests for Ziti service %v\n", serviceName)
    serve(listener, "ziti")
}

```

```
func main() {  
    if os.Getenv("DEBUG") == "true" {  
        pfxlog.GlobalInit(logrus.DebugLevel, pfxlog.DefaultOptions())  
        pfxlog.Logger().Debugf("debug enabled")  
    }  
  
    // Startup zitified server and plain http server  
    zitifiedServer()  
    // httpServer("localhost:8080")  
}
```

VM3

Kodu ben yazmadım. AI da yazmadı. SDK içerisinde hazır bir koddu.
AI bana sadece çalıştırmak için bir kaç trick verdi.

```
go zitifiedServer()  
    httpServer("localhost:8080")
```

bu dosyadaki kod parçasını

```
zitifiedServer()  
    // httpServer("localhost:8080")
```

bu şekilde değiştirerek çalıştırabileceğimi gösterdi.

Resmi SDK

<https://github.com/openziti/sdk-golang>

Çalıştırma Süreci

1. Adım

Önce `Staj görevi` dökümanındaki örnek sdk reposunu indiriyoruz

```
sudo apt install -y golang-go gcc git  
git clone https://github.com/openziti/sdk-golang.git
```

2. Adım

Yukarıda yazılan değişikliği uygulamak için inen repodaki

`~/sdk-golang/example/simple-server/simple-server.go` dosyasını açıyoruz.

3. Adım

Değişiklik yaptığımız go dosyasını build ediyoruz.

```
cd sdk-golang/example  
export ZITI_SDK_BUILD_DIR=$(pwd)/build  
mkdir -p $ZITI_SDK_BUILD_DIR  
go mod tidy  
go build -o build ./...  
ls build/
```

VM3

4. Adım

Build edilen dosyayı çalıştırmak

```
~/sdk-golang/example/build/simple-server ~/sdk-app.json sdk-zeroneSVC
```